

Modelagem, Implementação e Análise Comparativa de Estruturas em Árvore Especializadas: Trie, Patricia, Splay, Treap e KD-Tree

Rita Mariê Amaral Siqueira

Disciplina de Algoritmos e Estruturas de Dados II

Centro Federal de Educação Tecnológica de Minas Gerais

Resumo—Este trabalho apresenta o estudo, a implementação e a análise comparativa de cinco estruturas de dados hierárquicas especializadas – Trie, Árvore Patricia, Árvore Splay, Árvore Treap e KD-Tree – em contraste com as estruturas fundamentais de Árvore Binária de Busca (BST) e Árvore AVL. Para cada estrutura, discutem-se os fundamentos teóricos, as decisões de projeto adotadas na implementação, a complexidade assintótica das operações centrais e o comportamento experimental observado sobre conjuntos de dados de até 100.000 elementos, sob diferentes distribuições (aleatória, ordenada e decrescente). Os resultados evidenciam divergências relevantes entre o comportamento teórico esperado e o comportamento medido experimentalmente. Por fim, discutem-se aplicações práticas de cada estrutura, as principais dificuldades de implementação enfrentadas e as limitações observadas.

Index Terms—estruturas de dados, árvores balanceadas, Trie, Patricia, Splay Tree, Treap, KD-Tree, análise de complexidade

I. INTRODUÇÃO E FUNDAMENTAÇÃO TEÓRICA

A. Contextualização

Estruturas de dados hierárquicas em árvore constituem um dos pilares fundamentais para a organização eficiente de informação, permitindo operações de busca, inserção e remoção em tempo sublinear na maioria dos cenários práticos. Enquanto as Árvores Binárias de Busca (BST) e as Árvores AVL, estudadas em etapas anteriores da disciplina, oferecem soluções gerais para dados ordenáveis por comparação direta de chaves, diversos problemas computacionais possuem características específicas – chaves textuais com prefixos compartilhados, padrões de acesso não uniformes, ou dados multidimensionais – que motivam o uso de estruturas especializadas.

Este trabalho tem como objetivo estender o estudo de estruturas de árvore para cinco variantes especializadas: a Trie e a Árvore Patricia, voltadas à representação eficiente de conjuntos de strings; a Árvore Splay, uma BST autoajustável; a Árvore Treap, que combina propriedades de BST e heap; e a KD-Tree, voltada à indexação espacial multidimensional.

B. Trie (Árvore de Prefixos)

A Trie [2] é uma estrutura hierárquica em que cada aresta representa um caractere de uma chave, e o caminho da raiz até um nó marcado como final de palavra reconstrói a chave armazenada. Sua propriedade fundamental é o *compartilhamento de prefixos*: strings que compartilham um prefixo comum compartilham também o caminho correspondente na árvore,

o que garante busca, inserção e remoção em tempo $O(m)$, onde m é o comprimento da chave, independentemente do número total de chaves armazenadas.

Na implementação adotada, cada nó contém um vetor de 26 ponteiros (um por letra do alfabeto $\{a, \dots, z\}$) e um sinalizador booleano indicando se aquele nó representa o final de uma palavra válida. Essa escolha garante acesso ao filho correspondente a um caractere em tempo $O(1)$, ao custo de um consumo de memória fixo por nó, independentemente do número de filhos efetivamente utilizados.

C. Árvore Patricia

A Árvore Patricia, originalmente proposta por Morrison [3], é uma variante compacta da Trie: sequências de nós com um único filho são fundidas em uma única aresta, rotulada com a substring correspondente, em vez de um caractere isolado. Essa compactação reduz drasticamente o número de nós necessários para armazenar um conjunto de chaves com muitos prefixos compartilhados, mantendo a mesma complexidade assintótica de busca da Trie, porém com menor overhead de memória e menor profundidade de árvore.

A operação central que distingue a Patricia da Trie é o mecanismo de *divisão de prefixo* (*split*): ao inserir uma chave cujo prefixo diverge no meio do rótulo de uma aresta existente, um nó intermediário é criado no ponto exato de divergência, e o rótulo original é dividido em duas partes. O mecanismo inverso, a *fusão* (*merge*), ocorre durante a remoção, quando um nó interno passa a ter um único filho e deixa de ser necessário como ponto de ramificação, sendo fundido com esse filho.

D. Árvore Splay

A Árvore Splay, introduzida por Sleator e Tarjan [4], é uma BST autoajustável que não mantém uma invariante de balanceamento estrutural explícita (como altura ou fator de balanceamento), mas garante complexidade amortizada $O(\log n)$ por meio de uma heurística de reorganização: após cada acesso (busca, inserção ou remoção), o nó acessado é trazido à raiz por meio de uma sequência de rotações (*splay*), obedecendo três casos — *zig* (rotação simples, quando o nó é filho da raiz), *zig-zig* (duas rotações no mesmo sentido, quando nó, pai e avô estão alinhados) e *zig-zag* (duas rotações em sentidos opostos).

Essa heurística confere à estrutura a propriedade de *localidade temporal de referência*: elementos acessados recente-

mente tendem a permanecer próximos da raiz, tornando acessos subsequentes a esses mesmos elementos significativamente mais baratos.

E. Árvore Treap

A Treap [5] é uma estrutura híbrida que associa a cada nó, além da chave, uma prioridade gerada aleatoriamente no momento da inserção. A estrutura satisfaz simultaneamente a propriedade de BST em relação às chaves e a propriedade de *heap* (max-heap) em relação às prioridades. Como as prioridades são independentes e identicamente distribuídas, a forma resultante da árvore é equivalente, em distribuição, à de uma BST construída a partir de uma permutação aleatória das chaves – garantindo altura esperada $O(\log n)$ sem a necessidade de invariantes de balanceamento determinísticos, à custa de um balanceamento apenas *probabilístico*.

F. KD-Tree

A KD-Tree, proposta por Bentley [6], generaliza a BST para o espaço k -dimensional. Cada nó armazena um ponto com k coordenadas, e o critério de comparação utilizado para decidir a subárvore de inserção alterna ciclicamente entre as k dimensões conforme a profundidade do nó (eixo = profundidade mod k). Geometricamente, cada nó corresponde a um hiperplano que particiona o espaço em duas regiões. Essa estrutura permite buscas exatas, buscas por intervalo (*range queries*) e buscas por vizinho mais próximo de forma significativamente mais eficiente do que uma varredura linear, desde que a árvore permaneça razoavelmente balanceada.

G. Diferenças conceituais em relação a BST e AVL

As cinco estruturas estudadas divergem da BST e da AVL em pelo menos um dos seguintes aspectos: (i) o critério de comparação usado em cada nível (fixo em BST/AVL; alternante por dimensão na KD-Tree; por caractere na Trie/Patricia); (ii) o mecanismo de balanceamento (determinístico e estrutural na AVL; probabilístico na Treap; amortizado via heurística de acesso na Splay; compactação de caminho na Patricia); e (iii) a granularidade da chave armazenada (valor atômico em BST/AVL/Treap/Splay; caractere ou substring em Trie/Patricia; vetor de coordenadas na KD-Tree).

II. PROJETO E IMPLEMENTAÇÃO

A. Decisões de projeto comuns

Todas as sete estruturas foram implementadas em C++ seguindo um padrão arquitetural comum, visando consistência e comparabilidade experimental:

- **Recursão com retorno de ponteiro:** as operações de inserção e remoção seguem o padrão em que cada chamada recursiva retorna o ponteiro para a raiz (possivelmente atualizada) da subárvore processada, e o nível superior reatribui esse retorno ao ponteiro do filho correspondente. Esse padrão evita a necessidade de ponteiros para o nó pai, exceto na Árvore Splay, cujo mecanismo de rotação *bottom-up* [4] exige navegação ascendente explícita.

- **Rejeição de chaves duplicadas:** seguindo a propriedade formal de BST descrita por Cormen *et al.* [1] ($y.chave \leq x.chave$ à esquerda, $y.chave \geq x.chave$ à direita), e considerando que o próprio texto trata o tratamento de duplicatas como um problema em aberto com múltiplas soluções de custo variável, optou-se por não permitir chaves duplicadas em nenhuma das sete estruturas, simplificando a lógica de remoção.
- **Instrumentação determinística de métricas:** em vez de medir consumo de memória por meio de chamadas ao sistema operacional (sujeitas a ruído e overhead do alocador), cada estrutura mantém um contador interno de memória, incrementado em `sizeof(No)` (mais o conteúdo de estruturas de tamanho variável, como `std::string` e `std::vector`, quando aplicável) a cada criação de nó, e decrementado no momento exato de cada remoção.
- **Validação defensiva de entrada:** índices calculados a partir de caracteres (Trie/Patricia) ou dimensões (KD-Tree) são validados antes de indexar estruturas de tamanho fixo, prevenindo acessos fora dos limites de memória.

B. Árvore AVL

O nó da AVL contém chave, ponteiros para os filhos esquerdo e direito, e um campo adicional de *fator de balanceamento* (diferença entre a altura das subárvores direita e esquerda), atualizado incrementalmente a cada inserção/remoção em vez de recalculado por travessia completa da subárvore – decisão que reduz o custo de manutenção de $O(n)$ para $O(1)$ por nó ajustado.

A organização segue a propriedade padrão de BST (Seção II-C), acrescida da invariante estrutural $|fb| \leq 1$ para todo nó. A manutenção dessa invariante é feita por meio de rotações simples (à esquerda ou à direita) e duplas (esquerda-direita ou direita-esquerda), disparadas no caminho de retorno da recursão de inserção/remoção sempre que $|fb| > 1$ é detectado (Algoritmo 1).

Algorithm 1 Inserção – AVL (esqueleto)

```

1: function INSERIR(no, chave)
2:   if no = NULO then return CRIARNO(chave)
3:   end if
4:   if chave < no.chave then no.esq ← INSE-
      RIR(no.esq, chave)
5:   else if chave > no.chave then no.dir ← INSE-
      RIR(no.dir, chave)
6:   end if
7:   ATUALIZARFATORBALANCEAMENTO(no)
8:   return REBALANCEARSENECESSARIO(no) ▷ aplica
      rotação conforme o caso
9: end function

```

A remoção segue os mesmos três casos da BST convencional (folha, um filho, dois filhos com sucessor *in-order*), com a etapa adicional de rebalanceamento no caminho de retorno – diferença relevante em relação à inserção, pois uma única remoção pode disparar rotações em múltiplos níveis da árvore,

não apenas no ponto de desbalanceamento mais próximo da raiz.

C. Árvore Binária de Busca (BST)

O nó contém apenas chave e ponteiros para os filhos esquerdo e direito, sem qualquer campo de controle estrutural. A organização segue a propriedade formal $y.chave \leq x.chave$ para todo nó y na subárvore esquerda de x , e $y.chave \geq x.chave$ para a subárvore direita [1].

Por decisão de projeto, **a BST não implementa qualquer estratégia de manutenção de propriedades estruturais**: inserções e remoções alteram apenas a região necessária da árvore, sem verificação ou correção de altura. Essa ausência é proposital – serve como referência de comparação (*baseline*) para as demais estruturas balanceadas, e é a explicação teórica direta para a degeneração da árvore em uma lista ligada sob entrada ordenada, discutida na Seção IV e observada experimentalmente na Seção V.

A remoção contempla os três casos clássicos: nó folha (remoção direta), nó com um filho (substituição pelo filho), e nó com dois filhos (substituição pela chave do sucessor *in-order* – o mínimo da subárvore direita – seguida da remoção recursiva desse sucessor em sua posição original).

D. Trie e Patricia

O nó da Trie contém um vetor fixo de 26 ponteiros e um sinalizador de fim de palavra. O nó da Patricia substitui o caractere único por uma `std::string` de prefixo, mantendo o mesmo vetor de 26 ponteiros para indexação $O(1)$ do próximo caractere divergente. A inserção na Patricia calcula, a cada nó visitado, a posição de divergência entre o rótulo armazenado e o restante da chave a inserir; caso a divergência ocorra antes do fim do rótulo, aciona-se a operação de *split* (Algoritmo 2). A remoção aciona a operação inversa de *merge* sempre que um nó interno, após a remoção, passa a ter exatamente um filho e não representa mais o final de nenhuma palavra.

Algorithm 2 Divisão de prefixo (*split*) – Patricia

```

1: function SPLIT(no, posDivergencia)
2:   novoPai  $\leftarrow$  CRIARNO
3:   novoPai.prefixo  $\leftarrow$  no.prefixo[0 : posDivergencia]
4:   no.prefixo  $\leftarrow$  no.prefixo[posDivergencia :]
5:   idx  $\leftarrow$  no.prefixo[0] – 'a'
6:   novoPai.filhos[idx]  $\leftarrow$  no
7:   return novoPai
8: end function

```

Ambas as estruturas normalizam as chaves de entrada (conversão para minúsculas e remoção de acentuação) antes da inserção, restringindo o alfabeto suportado a $\{a, \dots, z\}$ – uma decisão de projeto documentada explicitamente para manter a simplicidade do vetor de 26 posições.

A organização das informações, em ambas as estruturas, é feita por *prefixo compartilhado*: cada caminho da raiz até

um nó marcado como fim de palavra representa uma chave inserida, e chaves com prefixos comuns compartilham os mesmos nós iniciais. Por decisão de projeto, **nenhuma das duas implementa rebalanceamento**: a profundidade máxima é limitada pelo comprimento da maior chave inserida, não pelo número de chaves – diferença fundamental em relação às árvores binárias balanceadas, discutida na Seção IV.

A busca por igualdade percorre a árvore caractere a caractere (Trie) ou comparando o prefixo armazenado em cada nó (Patricia), retornando sucesso apenas se o nó final estiver marcado como fim de palavra. Além da busca por igualdade, ambas implementam **busca por prefixo** (utilizada, por exemplo, em funcionalidades de autocompletar): a árvore é percorrida até o nó correspondente ao fim do prefixo consultado, e a partir dali um percurso completo da subárvore enumera todas as palavras que o compartilham.

Algorithm 3 Fusão de nós (*merge*) – Patricia

```

1: function MERGE(no)
2:   if no.fimDePalavra ou NUMFILHOS(no)  $\neq$  1 then
3:     return no ▷ nada a fundir
4:   end if
5:   filho  $\leftarrow$  UNICOFILHO(no)
6:   filho.prefixo  $\leftarrow$  no.prefixo + filho.prefixo
7:   return filho ▷ no é descartado; filho assume seu lugar
8: end function

```

E. Árvore Splay

A estrutura de dados Splay Tree foi implementada seguindo a abordagem Bottom-Up, conforme proposta no artigo seminal de Sleator e Tarjan (1985) [4]. O nó da Splay inclui um ponteiro adicional para o pai, necessário para a navegação ascendente do mecanismo de *splay*. As rotações simples (*zig/zag*) foram implementadas como funções genéricas parametrizadas apenas pelo nó a ser promovido, permitindo compor as rotações duplas (*zig-zig*, *zig-zag*) por meio de duas chamadas sucessivas às rotações simples. A operação de remoção segue o procedimento clássico: o nó a remover é levado à raiz via *splay*; em seguida, sua subárvore esquerda e direita são desconectadas, o predecessor da subárvore esquerda é elevado à raiz dessa subárvore (via novo *splay*), e a subárvore direita é reconectada como seu filho direito.

A organização das informações segue a propriedade padrão de BST por chave (Seção II-C); a diferença da Splay em relação às demais estruturas está inteiramente na operação de acesso, não na regra de ordenação.

A inserção segue a descida padrão de BST até a posição correta, seguida da promoção do nó recém-criado à raiz via *splay*. A busca segue o mesmo princípio: independentemente de a chave ser encontrada ou não, o último nó visitado na descida é promovido à raiz, de modo que buscas repetidas ao mesmo elemento (ou a elementos próximos) tornam-se progressivamente mais baratas – fundamento do custo amortizado $O(\log n)$ discutido na Seção IV.

O mecanismo de *splay* contempla três casos, aplicados repetidamente até o nó alvo alcançar a raiz: *zig* (o nó é filho da raiz; uma única rotação simples), *zig-zig* (nó, pai e avô alinhados no mesmo sentido; duas rotações simples no mesmo sentido) e *zig-zag* (nó e pai em sentidos opostos; duas rotações simples em sentidos opostos, equivalente a uma rotação dupla).

O percurso *in-order* produz a sequência ordenada das chaves, como em qualquer BST; não há operação específica adicional além do próprio mecanismo de *splay*, que pode ser interpretado como uma "operação de acesso com efeito colateral estrutural".

F. Árvore Treap

Cada nó armazena, além da chave, uma prioridade de 64 bits gerada por um gerador de números pseudoaleatórios (`std::mt19937`) no momento da criação, garantindo a propriedade de balanceamento probabilístico, seguindo as especificações do artigo de Seidel e Aragon (1996) [5]. A inserção segue a lógica de BST convencional durante a descida; no caminho de retorno da recursão, sempre que a prioridade de um filho excede a do próprio nó, aplica-se a rotação simples correspondente para restaurar a propriedade de heap. A remoção rotaciona o nó a ser removido para baixo, sempre em direção ao filho de maior prioridade, até que ele se torne uma folha, quando então é removido diretamente.

A organização das informações é dupla: a árvore é simultaneamente uma BST por chave e um *max-heap* por prioridade – isto é, para todo nó, sua prioridade é maior ou igual à de seus filhos, enquanto a chave respeita a ordenação usual de BST. É essa combinação que garante, probabilisticamente, altura esperada $O(\log n)$ sem qualquer verificação explícita de balanceamento.

A busca segue a descida padrão de BST por chave, ignorando completamente o campo de prioridade – este é utilizado exclusivamente durante inserção e remoção. O percurso *in-order* produz a sequência ordenada das chaves; um percurso por nível (*BFS*), por sua vez, evidenciaria a organização por prioridade (propriedade de heap).

G. KD-Tree

Optou-se por uma implementação genérica em número de dimensões, definida em tempo de execução (não via *templates* de C++), com as coordenadas armazenadas em um `std::vector<double>` de tamanho k . Cada nó armazena o eixo de comparação utilizado, calculado uma única vez em sua criação, evitando recomputação a partir da profundidade em operações subsequentes. A remoção de um nó com dois filhos substitui sua chave pelo mínimo da subárvore direita na dimensão do próprio nó (ou, na ausência de filho direito, pelo mínimo da subárvore esquerda, promovida à direita) – técnica descrita originalmente por Bentley [6], cuja correção foi verificada experimentalmente neste trabalho por meio de testes dirigidos a esse caso específico.

A inserção segue a descida padrão de BST, alternando a dimensão de comparação a cada nível conforme o eixo fixado em cada nó. Por decisão de projeto, a **KD-Tree não implementa**

rebalanceamento dinâmico: a ordem de inserção determina permanentemente a forma da árvore, o que a torna sensível a conjuntos de dados adversariais (e.g., pontos inseridos em ordem já particionada por eixo) – limitação discutida teoricamente na Seção IV e observada experimentalmente na Seção V.

Além da busca por igualdade (idêntica à BST, restrita à comparação na dimensão de cada nível), duas operações específicas da estrutura foram implementadas, por serem seu principal motivo de uso prático:

- **Busca por região**: retorna todos os pontos contidos em um hiper-retângulo dado, podando a recursão em subárvores cujo intervalo, na dimensão do nó, não intersecta a região de busca;
- **Vizinho mais próximo**: percorre a árvore priorizando o lado mais promissor (mais próximo do ponto de consulta na dimensão do nó atual) e poda a busca no lado oposto sempre que a distância até o hiperplano de corte for maior que a menor distância já encontrada.

Algorithm 4 Vizinho mais próximo – KD-Tree (esqueleto)

```

1: function VIZINHOMAISPROXIMO(no, ponto, melhor)
2:   if no = NULO then return melhor
3:   end if
4:   melhor  $\leftarrow$  ATUALIZARSEMAISPROXIMO(no, ponto, melhor)
5:   eixo  $\leftarrow$  no.eixo
6:   ladoPerto, ladoLonge  $\leftarrow$  DEFINIRLADOS(no, ponto, eixo)
7:   melhor  $\leftarrow$  VIZINHOMAISPROXIMO(ladoPerto, ponto, melhor)
8:   if  $|ponto[eixo] - no.coord[eixo]| < dist(melhor, ponto)$  then
9:     melhor  $\leftarrow$  VIZINHOMAISPROXIMO(ladoLonge, ponto, melhor)
10:  end if
11:  return melhor
12: end function

```

H. Módulo compartilhado de manipulação de arquivos

Uma classe utilitária centraliza a leitura parcial de datasets (interrompendo a leitura assim que o número de elementos solicitado é atingido, sem processar o arquivo integralmente) e a escrita padronizada dos resultados experimentais em formato CSV, evitando duplicação dessa lógica entre as sete estruturas.

III. DEMONSTRAÇÃO E RASTREAMENTO VISUAL

Para cada estrutura, foram geradas representações gráficas no formato DOT (Graphviz), capturando as operações realizadas em cada árvore, com foco em três momentos específicos: (i) o estado inicial após um conjunto de inserções; (ii) um estado intermediário, evidenciando o mecanismo estrutural específico da árvore; e (iii) o estado após uma operação de remoção.

A. Árvores numéricas (BST, AVL, Treap, Splay)

Utilizou-se a sequência de inserção $\{10, 5, 8, 16, -8, 56, 6\}$ como base comum às quatro estruturas, permitindo comparação direta do resultado estrutural sobre o mesmo conjunto de dados.

1) *BST*: A evolução da BST reflete a manutenção da propriedade de ordenação em três momentos, apresentados na

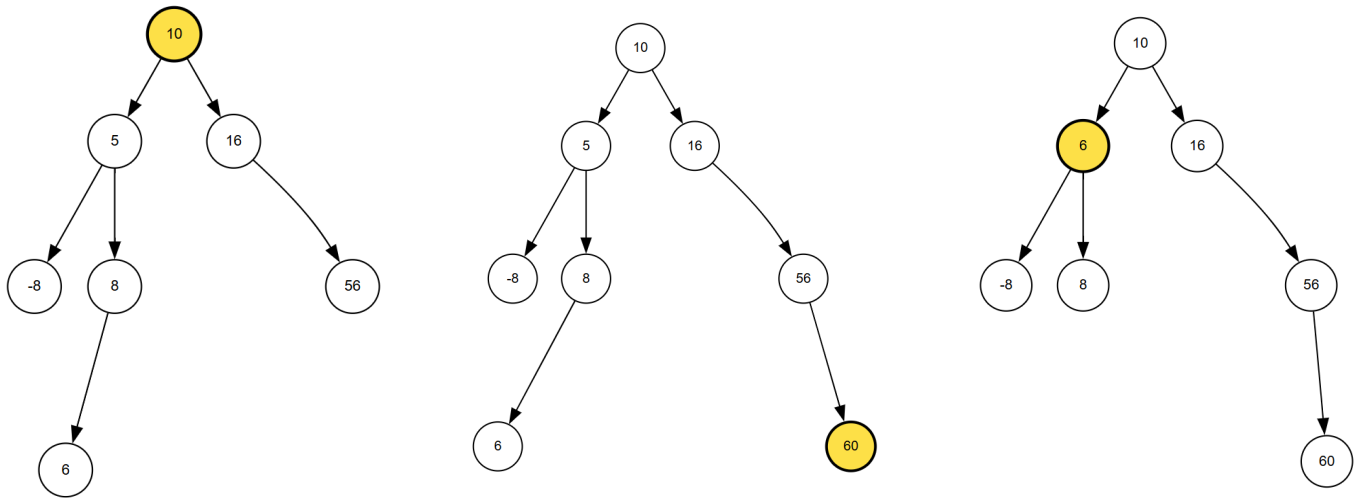


Figura 1: Evolução estrutural da Árvore Binária de Busca (BST): (a) configuração inicial com chaves base; (b) inserção demonstrando o posicionamento em folha; (c) reorganização após deleção de nó interno com dois filhos via sucessor in-ordem.

2) *AVL*: A demonstração da Árvore AVL ilustra o controle de altura por meio do Fator de Balanço ($FB \in \{-1, 0, 1\}$). O rastreamento evidencia três estados fundamentais na Figura 2: (i) o estado inicial, consolidado após as inserções da sequência base e ajustes automáticos por rotações simples; (ii) o estado intermediário, no qual a inserção das chaves 100 e 150 provoca

Figura 1: (i) a estrutura inicial gerada após as inserções da sequência base; (ii) a inserção da chave 60, demonstrando a busca e alocação descendente em nó folha à direita do 56; e (iii) a remoção do nó 5 (com dois filhos), resolvida via substituição pelo seu sucessor in-ordem (chave 6) para preservar a invariante da árvore.

o desbalanceamento temporário da subárvore direita ($FB = -2$), sendo corrigido via rotação para a esquerda; e (iii) a remoção da raiz (chave 8), demonstrando a substituição pelo seu sucessor in-ordem (chave 10) combinada à propagação do rebalanceamento ascendente nos nós ancestrais.

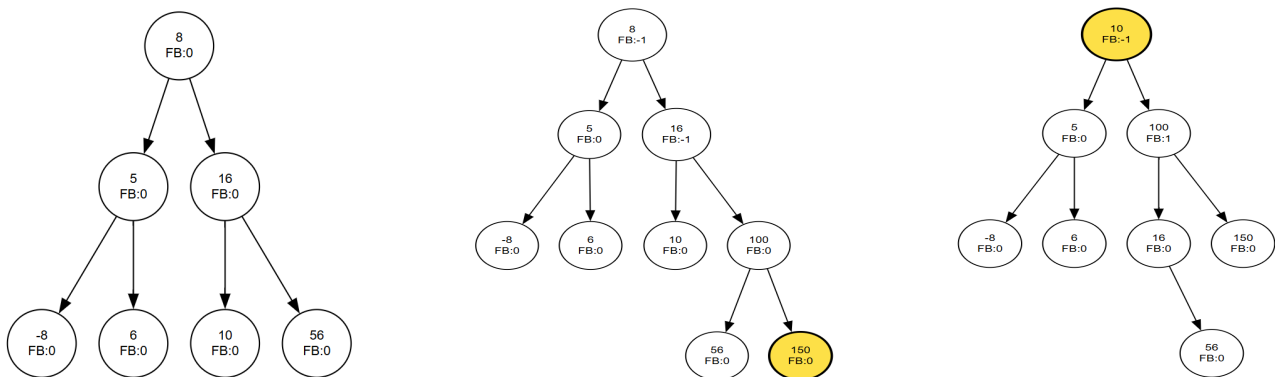


Figura 2: Evolução estrutural da Árvore AVL: (a) estado inicial mantendo o Fator de Balanço (FB); (b) inserção causando rotação para a esquerda; (c) remoção do nó raiz com rebalanceamento dos ancestrais.

3) *Treap*: A demonstração da Treap evidencia a manutenção simultânea das propriedades de Árvore Binária de Busca

(pelas chaves) e Max-Heap (pelas prioridades probabilísticas) na Figura 3: (i) a estrutura formada após as inserções base;

(ii) a inserção da chave 60 acompanhada de uma prioridade elevada ($P = 160$), que força rotações sucessivas para subir o nó até a raiz sem violar a ordenação das chaves; e (iii) a

remoção do nó 8, na qual o nó é rotacionado para baixo até se tornar folha antes de sua desalocação.

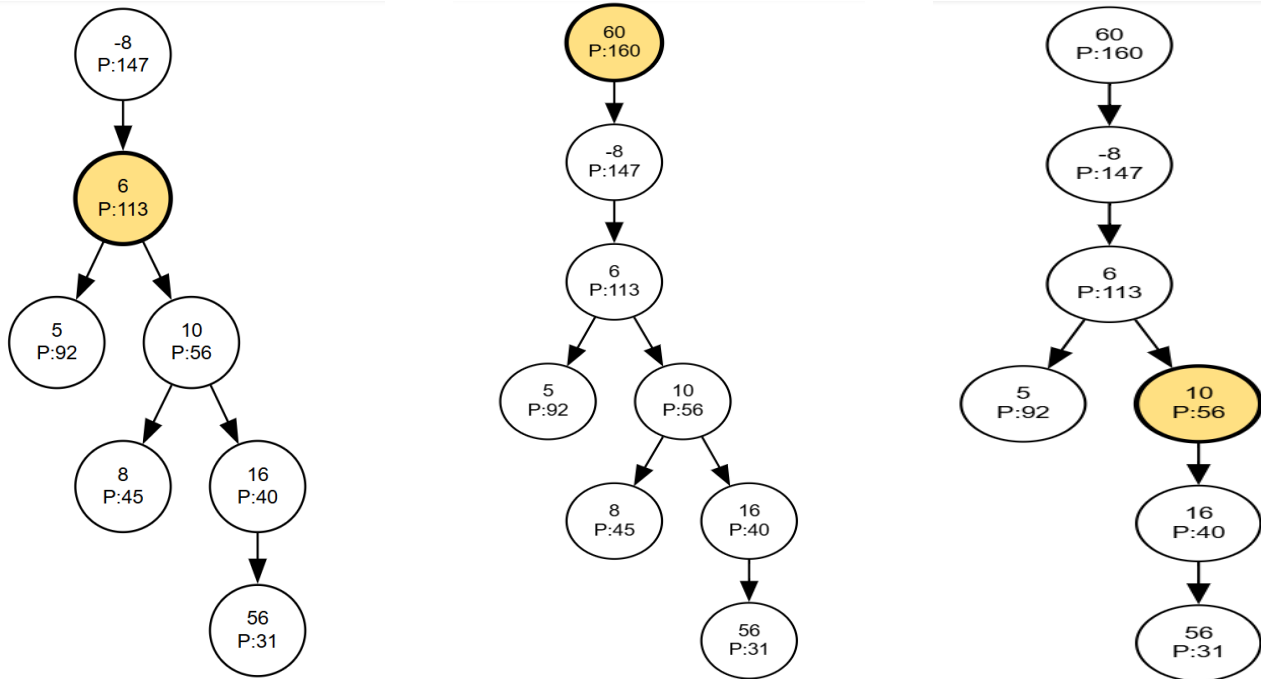


Figura 3: Evolução estrutural da Árvore Treap: (a) estrutura formada após as inserções base; (b) inserção da chave 60 (prioridade 160) com rotações sucessivas até a raiz; (c) remoção do nó 8 via rotação descendente até torná-lo folha.

4) *Splay*: A demonstração da Splay Tree evidencia o princípio de autoajuste Bottom-Up, no qual qualquer nó acessado é promovido à raiz por meio de uma sequência de rotações. O rastreamento apresenta três momentos-chave na Figura 4: (i) o estado inicial, no qual a última chave inserida (6) torna-se a raiz; (ii) o estado intermediário, em que a busca pela

chave -8 dispara rotações ascendentes para trazê-la ao topo da árvore; e (iii) a remoção da chave 16, demonstrando o procedimento padrão de trazer a chave alvo para a raiz via splay, realizar a partição em duas subárvores e unificá-las através do splay do maior elemento da subárvore esquerda (chave 10), promovendo-o como nova raiz.

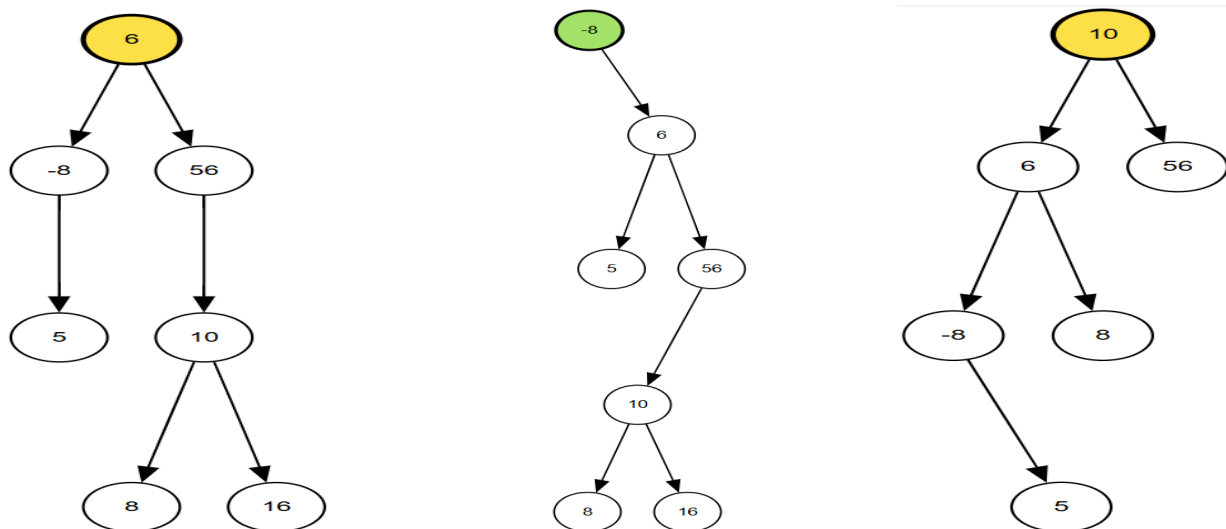


Figura 4: Evolução estrutural da Splay Tree: (a) configuração inicial após inserções base com a chave 6 na raiz; (b) rearranjo bottom-up pós-busca do nó -8; (c) remoção do nó 16 via splay, partição e fusão resultando na chave 10 como nova raiz.

B. Trie e Patricia

Utilizou-se o conjunto de palavras {*casa*, *casaco*, *carro*, *café*, *castelo*, *casulo*, *carroça*, *cão*, *coração*, *Ação*, *computacao*, *copo*}, escolhido por conter dois pares de palavras com prefixo comum (*cas-*, *cao-* e *car-*), o que evidencia tanto o compartilhamento de prefixos da Trie quanto a compactação da Patricia.

1) *Trie*: Para garantir a concisão e a clareza da exposição, optou-se por incluir no documento apenas a representação gráfica do estado final da estrutura (Figura 5). A inclusão de diagramas individuais para cada etapa intermediária resultaria em uma redundância visual excessiva, aumentando a extensão do texto sem acrescentar informações estruturais que não pudessem ser deduzidas a partir da topologia final e da análise lógica das operações.

No entanto, para efeitos de rastreo de execução e verificação do comportamento dos ponteiros da estrutura de dados, o processo de transformação da árvore ao longo das etapas ocorreu da seguinte forma:

- **Passo 1 — Inserção das palavras, com destaque para "casa", "casaco" e "castelo"**: A árvore é inicializada a partir de um nó raiz. A inserção da palavra "casa" cria uma sequência linear de quatro nós interligados pelos caracteres $c \rightarrow a \rightarrow s \rightarrow a$, em que o último nó é marcado como terminal. Em seguida, ao inserir "casaco", a estrutura reaproveita o prefixo comum "casa" até o quarto nó, estendendo a ramificação com as arestas $c \rightarrow o \rightarrow a$ até atingir um novo nó terminal. Por fim, a inserção de "castelo" reaproveita o prefixo "cas" até o terceiro nó; a

partir do nó do caractere *s*, cria-se uma nova ramificação divergente com a sequência de arestas $t \rightarrow e \rightarrow l \rightarrow o$, marcando o nó final do *o* como terminal.

- **Passo 2 — Inserção da palavra "computador"**: A partir do nó raiz, a busca pelo prefixo de "computador" encontra apenas a correspondência do primeiro caractere (*c*). Como o segundo caractere na árvore existente era o *a* (caminho para "casa"), a árvore realiza uma bifurcação no nó filho da raiz: o nó do *c* passa a ter dois filhos. Uma nova ramificação é criada a partir da aresta *o*, desencadeando a alocação sequencial em memória dos nós para a cadeia de caracteres restantes ($m \rightarrow p \rightarrow u \rightarrow t \rightarrow a \rightarrow d \rightarrow o \rightarrow r$), culminando em um nó terminal no último caractere.
- **Passo 3 — Remoção da palavra "castelo"**: A operação de remoção consulta a árvore até localizar o nó terminal correspondente ao último caractere de "castelo". A partir do nó *o*, o algoritmo executa a desalocação dos nós de forma ascendente (*bottom-up*). Como a subcadeia "telo" não é compartilhada por nenhuma outra palavra e nenhum dos seus nós intermediários é terminal, os nós referentes aos caracteres *o*, *l*, *e* e *t* são removidos da memória. A desalocação é interrompida ao atingir o nó associado ao caractere *s*, pois este possui outro nó filho ativo (a aresta *a* que dá seguimento à palavra "casa").

A Figura 5 reflete o resultado acumulado de todas estas operações, preservando as ramificações ativas de "casa", "casaco" e "computador".

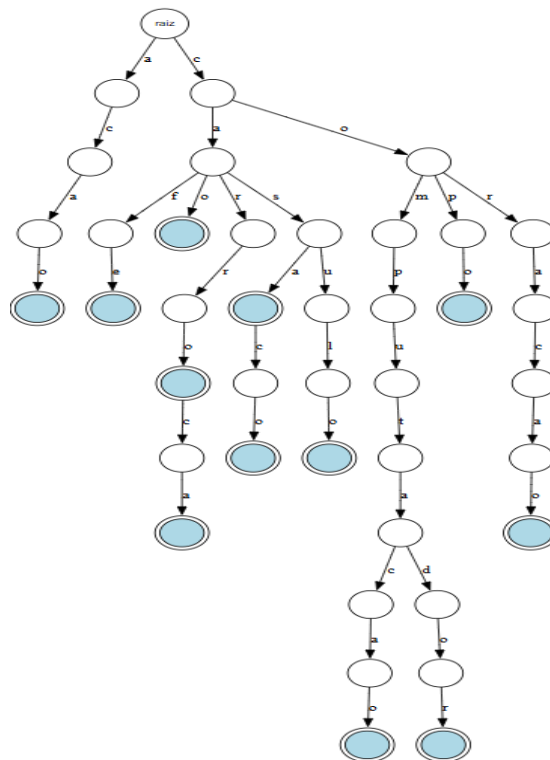


Figura 5: Estrutura final da Trie Tree

2) *Patrícia*: A demonstração da árvore PATRICIA evidencia a otimização de espaço sobre a Trie tradicional através da compressão de caminhos de filho único. O rastreamento apresenta três momentos cruciais na Figura 6: (i) a configuração inicial; (ii) a inserção da palavra "computador", que identifica o prefixo comum "mputa" sob o nó ancestral e divide

a aresta original para criar dois novos caminhos folha ("cao" e "dor"); e (iii) a remoção da palavra "castelo", na qual a folha "telo" é desalocada diretamente, preservando a propriedade de compactação ao evitar a existência de nós intermediários não ramificados.

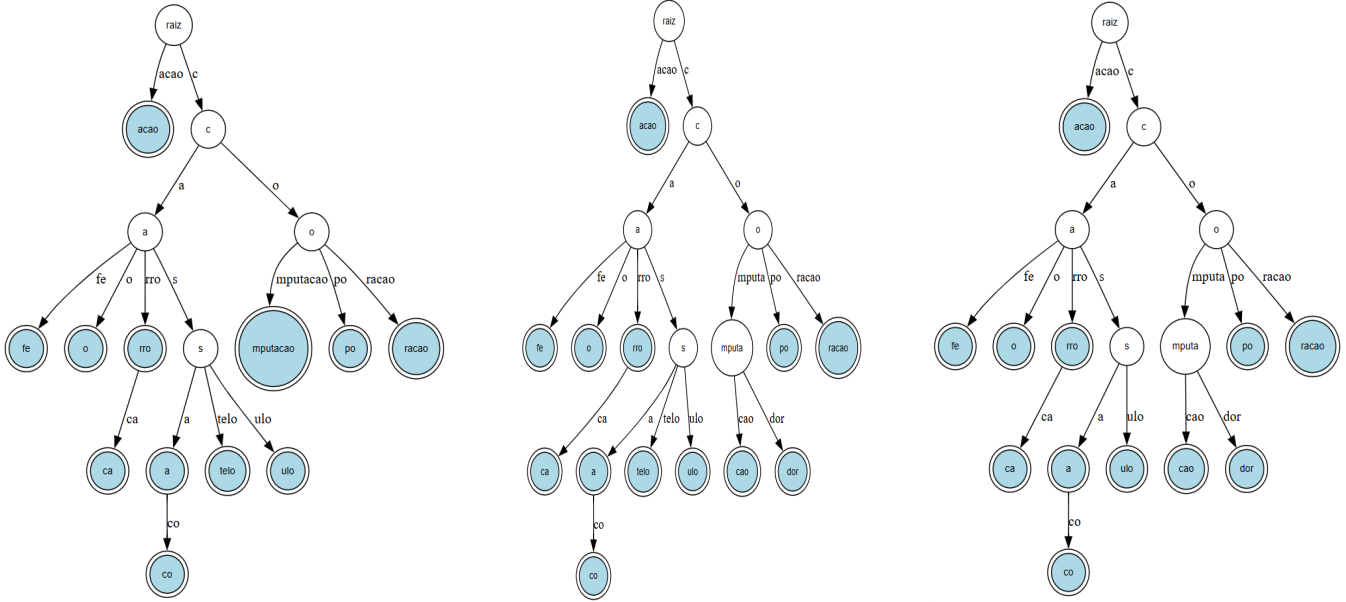


Figura 6: Evolução estrutural da Árvore PATRICIA: (a) compactação de sequências unárias em arestas; (b) cisão do nó "mputacao" para inserção da nova chave; (c) eliminação da folha "telo" mantendo a compactidade da árvore.

C. KD-Tree

A Figura 7 ilustra o rastreamento de execução da árvore k -d ($k = 2$) demonstrando o particionamento espacial em duas dimensões por meio de três etapas fundamentais: (i) a estrutura obtida após a inserção sequencial dos pontos 2D, na qual os nós alternam estritamente os eixos de corte (X e Y) a cada nível de profundidade; (ii) a inserção do ponto $(7, 13)$,

demonstrando a navegação pelos hiperplanos de corte ao longo da árvore até a alocação do nó na subárvore correspondente; e (iii) a remoção do nó de corte interno $(17, 15)$, detalhando a substituição adequada pelo nó candidato com o valor mínimo na coordenada Y pertencente à subárvore direita, preservando a consistência dos ponteiros e a propriedade de divisão do espaço.

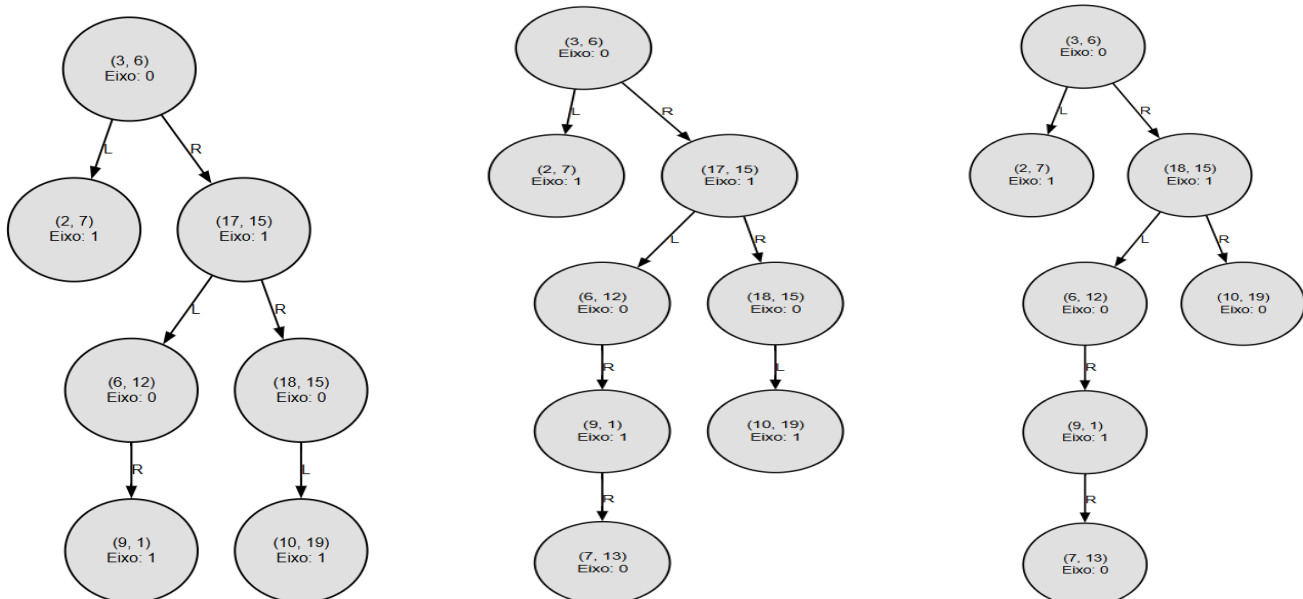


Figura 7: Rastreamento da árvore k -d ($k = 2$): (i) inserção e alternância de eixos; (ii) navegação no espaço 2D; e (iii) remoção de nó interno com reestruturação local..

IV. ANÁLISE DE COMPLEXIDADE E COMPARAÇÃO TEÓRICA

A Tabela I resume a complexidade assintótica das sete estruturas. Note-se que a distinção entre caso médio e pior é estendida também às operações de inserção e remoção nas estruturas sem garantia determinística de balanceamento (BST, Treap, KD-Tree), pelo mesmo motivo que já a justifica na busca: ambas as operações iniciam com uma busca pela posição correta, herdando seu comportamento assintótico antes de qualquer ajuste estrutural adicional. Cabem três observações analíticas adicionais.

Primeiro, a diferença fundamental entre BST e AVL não está na complexidade de *caso médio*, idêntica entre ambas, mas na garantia do *pior caso*: a BST admite degenerar em uma lista ligada sob sequências adversariais (e.g., entrada ordenada), enquanto a invariante de balanceamento da AVL ($|fb| \leq 1$ em todo nó) impede essa degeneração, à custa de rotações adicionais durante inserção e remoção.

Segundo, a Splay não mantém uma invariante estrutural instantânea – uma única operação pode, no pior caso, custar $O(n)$ (por exemplo, acessar o elemento mais profundo de uma árvore degenerada) – mas o Balance Theorem de Sleator e Tarjan [4] garante que o custo *amortizado* por operação, ao longo de uma sequência de m operações, é $O(\log n)$. Esse é o motivo pelo qual uma única medição isolada de tempo na Splay pode ser enganosa, mas o comportamento agregado sobre um conjunto de dados coincide com as demais estruturas balanceadas.

Terceiro, a complexidade da KD-Tree é fortemente dependente da dimensionalidade e da distribuição dos dados: a garantia de $O(\log n)$ pressupõe uma árvore razoavelmente balanceada, o que deixa de ser válido quando as k dimensões dos dados de entrada são fortemente correlacionadas – cenário explorado experimentalmente na Seção V. A construção por inserções sucessivas herda o mesmo comportamento médio/pior da busca ($O(n \log n) / O(n^2)$). Já as operações específicas da estrutura – busca do vizinho mais próximo e busca por região – têm custo fortemente dependente da geometria dos dados: no caso médio, a poda de subárvores inteiras mantém o custo próximo de $O(\log n)$ para o vizinho mais próximo e $O(n^{1-1/k} + r)$ para buscas por região (sendo r o número de resultados retornados); no pior caso (poda ineficaz, tipicamente sob dados de alta dimensionalidade ou fortemente correlacionados), ambas degradam para $O(n)$ [6].

Em relação à Trie e à Patricia, a complexidade das operações depende do comprimento da chave m , não do número de elementos armazenados n – uma vantagem assintótica sobre as estruturas baseadas em comparação quando $m \ll \log n$, e uma desvantagem quando m é grande. A diferença essencial entre as duas reside no fator de espaço: a Trie aloca um nó (e seu vetor de 26 ponteiros) por caractere, enquanto a Patricia aloca um nó apenas em pontos de ramificação efetiva, reduzindo

o número de nós de $O(n \cdot m)$, no pior caso, para $O(n)$. A construção de ambas custa $O(\sum m_i)$, onde $\sum m_i$ é a soma dos comprimentos de todas as chaves inseridas – cada caractere é processado exatamente uma vez, independentemente da ordem de inserção, o que contrasta com a sensibilidade à ordem observada nas estruturas baseadas em comparação. A operação específica de ambas – busca por prefixo, utilizada por exemplo em funcionalidades de autocompletar – custa $O(m + r)$, onde m é o comprimento do prefixo consultado e r o número de palavras retornadas. Por fim, em termos de espaço da Patricia: além do fator $O(n \cdot |\Sigma|)$ relativo aos vetores de ponteiros, cada nó também armazena a *string* de prefixo correspondente (Seção II), cujo armazenamento total é limitado por $O(\sum m_i)$ – termo que pode dominar o custo espacial quando as chaves são longas, mesmo com o número de nós reduzido pela compressão de caminhos.

V. METODOLOGIA EXPERIMENTAL E RESULTADOS

A. Metodologia

Os experimentos foram conduzidos sobre conjuntos de dados gerados sinteticamente, com tamanhos $N \in \{100, 1\,000, 10\,000, 50\,000, 100\,000\}$. Para as estruturas numéricas (BST, AVL, Splay, Treap), utilizaram-se três distribuições: *aleatória* (permutação uniforme, sem repetição), *ordenada* (crescente) e *decrecente* – as duas últimas representando cenários adversariais clássicos para estruturas não balanceadas. Para Trie e Patricia, geraram-se palavras com prefixos compartilhados em múltiplos níveis, também nas três distribuições (por ordem alfabética das palavras geradas). Para a KD-Tree, utilizaram-se três padrões espaciais: pontos uniformemente aleatórios, pontos organizados em agrupamentos gaussianos (*clusters*) e pontos dispostos ao longo de uma diagonal no espaço (cenário adversarial multidimensional).

Para cada combinação de estrutura, dataset e tamanho, mediu-se a média dos 10 testes: tempo total de execução (buscando-se a totalidade dos elementos inseridos e removendo-se metade deles, selecionados aleatoriamente), número de comparações, número de rotações (quando aplicável), altura final e consumo de memória, este último instrumentado internamente de forma determinística (Seção II-A).

Durante a execução preliminar dos experimentos, identificou-se que a combinação BST + dataset ordenado, KD-Tree + dataset diagonal e KD-Tree + remoção de 50% dos elementos, para $N \geq 50\,000$, produziam tempos de execução da ordem de dezenas de segundos a minutos por medição isolada – consistente com o comportamento $O(n^2)$ discutido na Seção IV. Optou-se por interromper essas medições quando o conjunto de entradas atingisse o limite de 8000 dados e quando a KD-Tree atingisse 1000 remoções. Adicionalmente, variou-se a dimensionalidade do espaço, $k \in \{2, 3, 4\}$, para avaliar experimentalmente o efeito da maldição da dimensionalidade discutida na Seção IV.

Tabela I: Complexidade assintótica das operações centrais

Estrutura	Busca (méd.)	Busca (pior)	Inserção (méd./pior)	Remoção (méd./pior)	Construção	Operação específica	Espaço
BST	$O(\log n)$	$O(n)$	$O(\log n) / O(n)$	$O(\log n) / O(n)$	$O(n \log n) / O(n^2)$	–	$O(n)$
AVL	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n \log n)$	Rotação: $O(1)$	$O(n)$
Treap	$O(\log n)$	$O(n)$ (raro)	$O(\log n) / O(n)$	$O(\log n) / O(n)$	$O(n \log n) / O(n^2)$	–	$O(n)$
Splay	$O(\log n)^a$	$O(n)$	$O(\log n)^a$	$O(\log n)^a$	$O(n \log n)^a$	–	$O(n)$
Trie	$O(m)$	$O(m)$	$O(m)$	$O(m)$	$O(\sum m_i)$	Prefixo: $O(m+r)$	$O(n \cdot m \cdot \Sigma)$
Patricia	$O(m)$	$O(m)$	$O(m)$	$O(m)$	$O(\sum m_i)$	Prefixo: $O(m+r)$	$O(n \cdot \Sigma + \sum m_i)$
KD-Tree (k -dim)	$O(\log n)$	$O(n)$	$O(\log n) / O(n)$	$O(\log n)^b / O(n^{1-1/k})^c$	$O(n \log n) / O(n^2)$	NN: $O(\log n)/O(n)$; Região: $O(n^{1-1/k}+r)$	$O(n)$

m = comprimento da chave; r = número de resultados retornados; $|\Sigma|$ = tamanho do alfabeto; n = número de elementos. ^a Complexidade amortizada (não se aplica ao pior caso de uma operação isolada, coluna "Busca (pior)"). ^b Remoção de nó folha ou com um filho. ^c Remoção de nó interno com dois filhos (busca de mínimo em 1 dimensão).

B. Resultados

A Tabela II consolida os tempos médios das operações fundamentais, refletindo a teoria das estruturas analisadas:

- **Árvores Binárias de Busca (BST, AVL, Splay e Treap):** As estruturas balanceadas superaram a BST em busca. A AVL obteve o menor tempo de busca (147,25 ns), graças ao rigoroso controle de altura. A Treap apresentou desempenho próximo (166,12 ns na busca), validando o uso de prioridades aleatórias. A Splay destacou-se na inserção mais rápida (246,04 ns) devido às rotações locais de *splaying*, enquanto a BST exibiu os maiores tempos de inserção (760,55 ns) e busca (369,24 ns) devido à ausência de rebalanceamento.
- **Árvores de Prefixo (Trie e Patricia):** Apresentaram custos na escala de microssegundos ($\approx 1,5\mu s - 2,1\mu s$), superiores aos das árvores numéricas por conta da complexidade $O(K)$ e do acesso a múltiplos ponteiros por caractere. A Trie foi mais eficiente que a Patricia (inserção: 1782,67 ns vs. 2138,48 ns), pois as operações de *bit-skipping* e checagem de chaves da Patricia introduzem um *overhead* instrucional.
- **Estrutura Multidimensional (KD-Tree):** A KD-Tree ($K = 2$) registrou tempos superiores de inserção (6941,76 ns) e busca (4944,62 ns) devido à alternância de eixos. A remoção foi a operação mais custosa do estudo (54117,26 ns), exigindo a busca recursiva do nó mínimo/máximo no eixo de corte e a reestruturação da subárvore.

Tabela II: Consolidação geral do desempenho médio observado (todas as distribuições e tamanhos de entrada agregados)

Estrutura	Complexidade	Inserção (ns)	Busca (ns)	Remoção (ns)
BST	$O(h) / O(N)$	760,55	369,24	197,71
AVL	$O(\log N)$	288,04	147,25	250,10
Splay	$O(\log N)$ amortiz.	246,04	273,59	347,04
Treap	$O(\log N)$ med.	276,87	166,12	272,30
Trie	$O(K)$	1782,67	1510,33	1815,22
Patricia	$O(K)$	2138,48	1779,40	1983,99
KDTree	$O(\log N) / O(K)$	6941,76	4944,62	54117,26

A Figura 8 apresenta o tempo de inserção para o dataset aleatório. A BST obteve o menor tempo em toda a faixa de N , superando as estruturas balanceadas. Como a entrada aleatória garante o caso médio $O(\log n)$, a BST evita o *overhead* de checagem e rotações presente na AVL, Treap e Splay, evidenciando o custo fixo de manutenção do balanceamento.

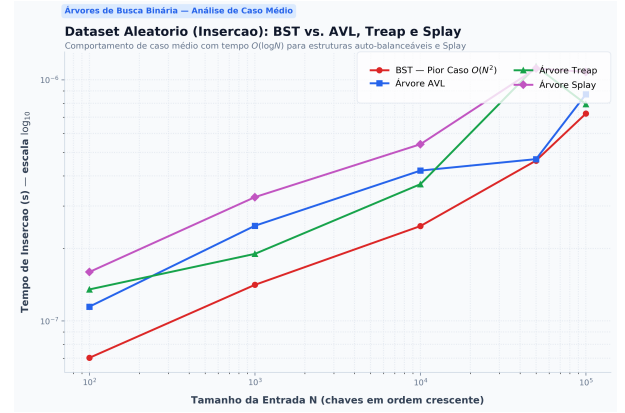


Figura 8: Caso médio de inserção – dataset aleatório.

Note-se que a Tabela II agrega todos os datasets, incluindo o cenário ordenado (adversarial para a BST); ao isolar apenas o dataset aleatório, como na Figura 8, o comportamento se inverte.

A Árvore Splay apresentou o maior tempo entre as quatro estruturas, pois promove o nó inserido até a raiz via *splay* incondicionalmente a cada operação, gerando mais rotações do que a AVL e a Treap.

Nota-se ainda que, entre $N = 10^2$ e $N = 10^5$, o tempo de inserção cresceu de 10 a 15 vezes, superando o fator teórico esperado ($\approx 2,5$). Essa divergência decorre da perda de localidade de referência na memória principal e do aumento de *cache misses* à medida que a árvore se expande, custo não medido pelo modelo assintótico puro.

A Figura 9 ilustra o comportamento temporal de inserção para uma entrada estritamente ordenada (pior caso). O gráfico evidencia de forma clara a vulnerabilidade da BST convencional, cujo tempo de execução cresce exponencialmente em escala logarítmica até interromper os testes por desbordamento, confirmando a sua degeneração para uma lista encadeada com complexidade acumulada $O(N^2)$.

Em contrapartida, as estruturas autobalanceadas sustentaram tempos de execução na ordem de 10^{-7} s ao longo de toda a variação de N , preservando o limite assintótico $O(N \log N)$. A Árvore Splay obteve o melhor desempenho absoluto sob inserção ordenada; como as chaves são inseridas em ordem crescente, as operações consecutivas de *splaying* realizam rotações à esquerda que mantêm a altura da árvore extremamente

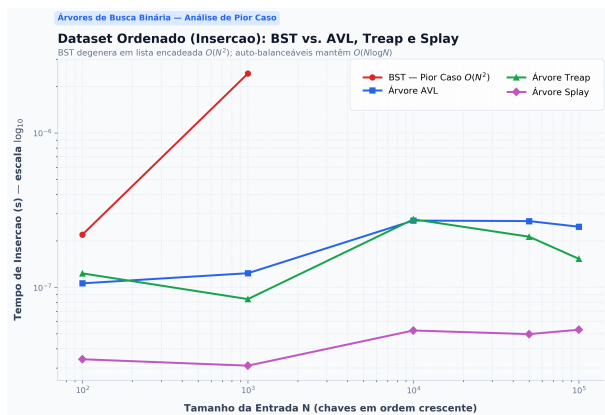


Figura 9: Pior caso de inserção – dataset ordenado.

reduzida e próxima de $O(1)$ amortizado para esse padrão de acesso específico. A Treap e a AVL mantiveram curvas estáveis e equivalentes, comprovando a eficácia do balanceamento estocástico por prioridades (Treap) e do rigoroso controle de altura por fatores de balanceamento (AVL) diante de entradas em ordem crescente.

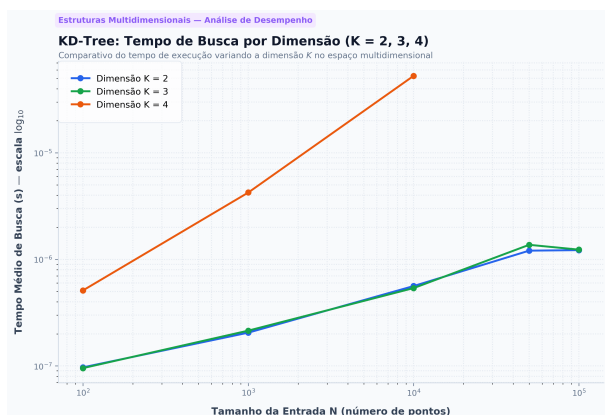


Figura 10: Tempo de Busca por Dimensão

A Figura 10 apresenta a variação do tempo médio de busca na KD-Tree ao aumentar a dimensão do espaço ($K = 2, 3, 4$) em função do número de pontos N . Observa-se que o desempenho para $K = 2$ e $K = 3$ é praticamente idêntico ao longo de toda a escala de N , mantendo tempos de busca reduzidos ($\approx 10^{-7}$ s a 10^{-6} s). Esse comportamento confirma a eficiência da partição espacial em poucas dimensões, em que a poda de subárvores durante a travessia permanece altamente efetiva.

Em contrapartida, a transição para $K = 4$ resulta num aumento substancial do tempo de busca em todas as ordens de magnitude de N , evidenciando a degradação do desempenho conhecida como a "maldição da dimensionalidade" (*curse of dimensionality*). Com o aumento de K , a eficácia dos hiperplanos de corte diminui, forçando o algoritmo de busca a explorar múltiplos ramos e aproxima-se de uma busca exaustiva de complexidade $O(N)$ à medida que a dimensão do espaço

cresce.

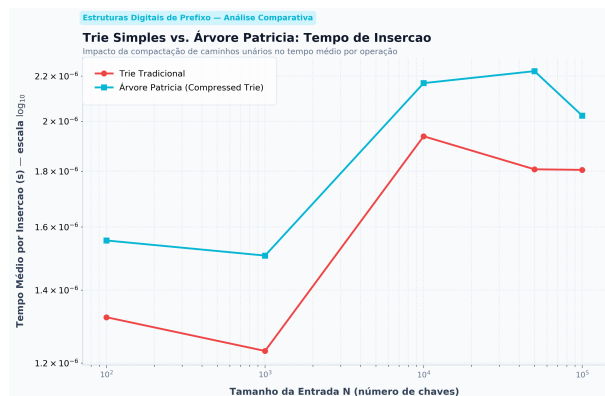


Figura 11: Trie vs Patricia - Tempo de Inserção com dataset ordenado

A Figura 11 apresenta a comparação do tempo médio de inserção por chave entre a Trie Tradicional e a Árvore Patricia (*Compressed Trie*) em função da dimensão do conjunto de dados N . O gráfico demonstra que a Trie Tradicional obtém tempos de inserção consistentemente inferiores aos da Árvore Patricia ao longo de toda a escala de N , mantendo-se na ordem dos $1,2 \times 10^{-6}$ s a $1,9 \times 10^{-6}$ s.

Embora a Árvore Patricia otimize substancialmente o consumo de memória ao compactar caminhos unários e eliminar nós redundantes com um único filho, essa compactação introduz um custo computacional adicional por inserção. O processo de inserção na Patricia exige a manipulação explícita de índices de bits (*bit-skipping*), a verificação e divisão de nós comprimidos existente e uma comparação final completa da chave na folha. Esse *overhead* instrucional e de lógica de branching na CPU supera, em termos de tempo de execução, a simples travessia por ponteiros diretos efetuada pela Trie Tradicional. Ambas as estruturas estabilizam os tempos médios a partir de $N = 10^4$, confirmando que a complexidade de inserção depende essencialmente do comprimento médio das chaves $O(K)$ e não da quantidade total de elementos N .

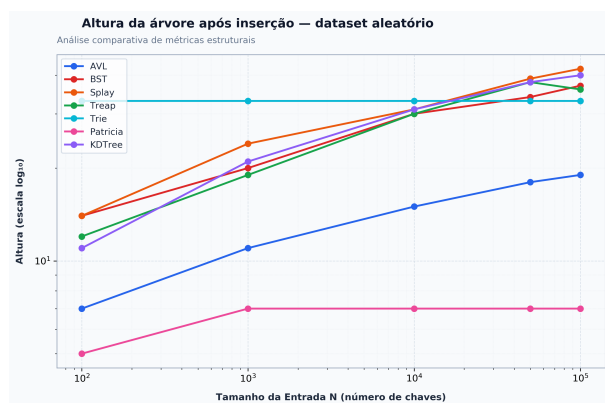


Figura 12: Comparação entre altura das árvores

A Figura 12 apresenta a evolução da altura final das diferentes estruturas de dados em função do tamanho do conjunto

de dados N em escala logarítmica. A análise revela comportamentos estruturais e teóricos distintos entre as famílias de árvores:

- **Árvores de Prefixo (Trie e Patricia):** A árvore Patricia apresentou a menor altura absoluta entre todas as estruturas avaliadas, estabilizando-se em valores reduzidos para $N \geq 10^3$. Esse resultado evidencia a eficácia da compactação de caminhos unários, limitando a altura estritamente ao número de bits/caracteres necessários para diferenciar as chaves. Por sua vez, a Trie convencional manteve uma altura rigorosamente constante em toda a faixa de N , uma vez que a sua altura máxima é estritamente limitada pelo comprimento fixo das chaves (K), independentemente da quantidade de elementos inseridos.
- **Árvores Autobalanceadas vs. BST:** A Árvore AVL exibiu o crescimento de altura mais controlado e eficiente entre as estruturas baseadas em comparações, mantendo valores significativamente inferiores aos da BST, Treap e Splay. Essa menor altura reflete o critério rigoroso de balanceamento da AVL ($h \leq 1,44 \log_2 N$), garantindo caminhos de busca otimizados.
- **BST, Treap, Splay e KD-Tree:** Sob entrada aleatória, a BST convencional, a Treap e a KD-Tree apresentaram curvas de crescimento de altura muito próximas entre si, acompanhando a evolução logarítmica esperada $O(\log N)$ no caso médio. A Árvore Splay registrou a maior altura dentre as estruturas de busca binária; embora garanta complexidade $O(\log N)$ amortizada, a sua reestruturação dinâmica focada no nó recém-acessado permite que ramos não visitados permaneçam temporariamente mais profundos do que numa árvore rigidamente balanceada por altura.

C. Discussão: Comportamento Assintótico vs. Prática Empírica

A análise conjunta dos experimentos revela um elevado grau de convergência entre as previsões teóricas e o desempenho empírico observado nas estruturas estudadas. De modo geral, o modelo de complexidade assintótica $O(f(N))$ provou ser um preditor altamente confiável do comportamento das operações de inserção, busca e remoção:

- **Convergências Principais:** As árvores autobalanceadas (AVL, Treap e Splay) confirmaram a contenção efetiva da degradação estrutural em cenários adversariais (dados ordenados), mantendo complexidade $O(\log N)$ onde a BST degenera para $O(N^2)$. Nas estruturas de prefixo (Trie e Patricia), a estabilização dos tempos em instâncias de grande porte ($N \geq 10^4$) comprovou empiricamente que o custo operacional depende estritamente do comprimento das chaves $O(K)$, e não da cardinalidade do conjunto de dados N . Da mesma forma, a KD-Tree demonstrou a eficiência da partição espacial em baixa dimensão ($K = 2, 3$), ao mesmo tempo em que a acentuada elevação do tempo para $K = 4$ evidenciou de forma clara a "maldição da dimensionalidade".

- **Divergências Observadas e Custo de Memória:** A principal divergência entre a teoria pura e a prática ocorreu na taxa de crescimento temporal sob grande volume de dados. Enquanto o modelo assintótico assume custo constante por comparação ou ponteiro, a execução física em hardware real sofre impacto severo da hierarquia de memória. À medida que N cresce, a perda de localidade de referência e o aumento expressivo de *cache misses* na RAM fazem com que o tempo de execução cresça ligeiramente acima da taxa logarítmica ideal. Além disso, a comparação entre Trie e Patricia demonstrou que otimizações puramente estruturais para redução de memória podem introduzir *overheads* computacionais (como *bit-skipping* e cisão de nós) que elevam o tempo de CPU por inserção.

Com o intuito de preservar a concisão visual e respeitar os limites de espaçamento da publicação, foram apresentados no corpo deste artigo apenas os gráficos mais representativos para cada classe de experimento. A suíte completa de gráficos gerados, contemplando todas as variações de parâmetros, métricas de memória e detalhamentos por operação, encontra-se publicamente disponível no repositório do projeto no GitHub [7].

VI. APLICAÇÕES, ANÁLISE CRÍTICA E DISCUSSÃO

A. Aplicações implementadas

Para cada estrutura, implementou-se uma aplicação relevante, exceto para BST e AVL, reaproveitando a lógica já validada nas operações centrais:

- **BST – Tabela de símbolos em protótipos e interpretadores:** em cenários de prototipagem rápida ou interpretadores simples, nos quais o padrão de inserção das chaves (identificadores, variáveis) é predominantemente aleatório e não adversarial, a BST oferece desempenho de caso médio equivalente ao das estruturas balanceadas (Seção V), com uma implementação sensivelmente mais simples e sem o overhead constante de verificação estrutural a cada operação – adequação que depende criticamente da ausência de padrões de entrada ordenados, cenário em que a estrutura deve ser evitada.
- **AVL – Índices em sistemas com exigência de latência previsível:** em bancos de dados em memória, sistemas de tempo real e agendadores, a garantia determinística de altura $O(\log n)$ da AVL evita os picos de latência que uma BST comum poderia produzir sob sequências de entrada desfavoráveis (Seção V). A adequação da AVL nesses cenários decorre diretamente de sua invariante estrutural rígida: ao contrário de garantias probabilísticas (Treap) ou amortizadas (Splay), o pior caso de uma única operação na AVL é conhecido e limitado, propriedade essencial quando a previsibilidade individual de cada operação importa mais que o desempenho agregado.
- **Trie – Autocompletar:** dado um prefixo, o algoritmo localiza o nó correspondente ao final do prefixo digitado (reaproveitando a lógica de descida por caractere) e realiza uma travessia em profundidade a partir desse nó,

coletando todas as palavras completas na subárvore – incluindo corretamente o caso em que o próprio prefixo é também uma palavra válida.

- **Patricia – Correspondência de maior prefixo (*longest prefix match*):** análoga ao problema de roteamento IP, em que se busca, entre as chaves armazenadas, o prefixo mais longo que corresponde a uma consulta.
- **Splay – Simulação de cache com localidade temporal:** simulação de 10.000 acessos sobre uma população de 100.000 elementos, seguindo um padrão 80/20 (80% dos acessos concentrados em 10% dos elementos), demonstrando experimentalmente a convergência dos elementos mais acessados para posições próximas da raiz.
- **Treap – Estrutura de prioridade com desempate probabilístico:** a prioridade aleatória associada a cada chave é utilizada como critério natural de desempate em cenários de fila com múltiplos elementos de mesma prioridade lógica.
- **KD-Tree – Busca espacial:** implementação de busca por vizinho mais próximo e busca por intervalo retangular (*range query*), aplicáveis a problemas de sistemas de informação geográfica e reconhecimento de padrões.

B. Vantagens e limitações observadas

Além das limitações para Trie/Patricia (restrição de alfabeto) e KD-Tree (sensibilidade a dados correlacionados (Seção VI-F)), os experimentos da Seção V evidenciam vantagens e limitações específicas também para as quatro estruturas de árvore binária. A BST tem como vantagem a simplicidade de implementação e o menor custo constante por operação (Seção V-B), mas sua limitação é severa e unilateral: nenhuma garantia é oferecida contra entradas adversariais, e sua adequação depende inteiramente de suposições sobre a distribuição dos dados de entrada que nem sempre podem ser verificadas em tempo de execução. A AVL tem como vantagem a garantia determinística de pior caso, mas paga por isso com maior custo constante em todas as operações, mesmo quando o rebalanceamento não é estritamente necessário – um custo fixo que se mostrou, na Seção V, superior ao de estruturas com garantias mais fracas (Treap) mesmo sob entrada não ordenada. A Splay tem como vantagem a adaptação automática a padrões de acesso com localidade temporal (demonstrada experimentalmente na aplicação de cache, Seção VI-A), mas essa mesma característica é uma limitação sob padrões de acesso uniformes, nos quais o custo de reorganização constante não traz benefício algum e a torna, na prática, a mais lenta das quatro estruturas balanceadas (Seção V). A Treap tem como vantagem a simplicidade de implementação (não exige lógica de rotação condicionada a múltiplos casos, como a AVL, nem navegação ascendente, como a Splay), mas sua garantia é apenas probabilística, dependendo da qualidade do gerador de números aleatórios utilizado – uma dependência ausente nas demais estruturas balanceadas.

C. Dificuldades de implementação

A remoção em estruturas com ponteiro para o nó pai (Splay) exigiu atenção redobrada à sincronização de três ponteiros a cada rotação (nó promovido, pai, e o filho que troca de subárvore), sendo a fonte mais frequente de erros durante o desenvolvimento. Na Patricia, a lógica de *merge* pós-remoção revelou-se a operação mais sutil de todo o trabalho: um erro de lógica na condição de proteção da raiz produzia uma árvore que nunca se tornava genuinamente vazia após a remoção do último elemento, apesar de reportar corretamente a remoção de cada chave individualmente – um defeito que só se manifestou em testes que esvaziavam a estrutura por completo, evidenciando a importância de casos de teste que cubram cenários extremos, não apenas o caso médio de uso.

Na KD-Tree, a validação de que a técnica de remoção (substituição pelo mínimo da subárvore direita, ou promoção da subárvore esquerda na ausência de filho direito) preserva a invariante espacial exigiu verificação cuidadosa: embora a intuição inicial sugerisse que mover uma subárvore inteira de um lado para o outro poderia violar a propriedade de particionamento, a demonstração de que o valor promovido é, por definição, o mínimo de toda a subárvore movida garante que a invariante permanece válida.

Comparando o esforço de desenvolvimento entre as sete estruturas, a Patricia foi a que exigiu maior tempo de depuração relativo à sua complexidade de especificação: embora a lógica de *split/merge* seja curta (poucas linhas de código, Seção II), o comportamento incorreto só se manifestava em cenários específicos de esvaziamento completo da estrutura, exigindo uma bateria de testes dirigidos que não foi necessária, no mesmo grau, para nenhuma das demais estruturas – ilustrando que a complexidade de manutenção de uma estrutura de dados não é proporcional apenas ao tamanho de sua implementação, mas também à sutileza de suas invariantes internas.

D. Divergências entre comportamento teórico e experimental

De modo geral, os resultados experimentais da Seção V confirmam qualitativamente as previsões assintóticas da Seção IV: estruturas com garantia determinística ou probabilística de balanceamento mantiveram-se estáveis sob todos os padrões de entrada testados, enquanto BST e KD-Tree degradaram sob os respectivos casos adversariais, na magnitude prevista teoricamente. Duas divergências quantitativas, no entanto, merecem destaque crítico. A primeira é o crescimento do tempo de execução da BST e da AVL sob entrada aleatória superior ao previsto apenas pela contagem de comparações: entre $N = 10^2$ e $N = 10^5$, a teoria prevê um aumento de custo por operação de aproximadamente $2,5 \times$ (razão entre $\log_2 N$ nos dois extremos), mas observou-se um aumento de 10 a 15 vezes no tempo medido – diferença atribuída ao custo de acesso à memória principal, não capturado pelo modelo assintótico baseado unicamente em número de comparações, à medida que a árvore deixa de caber inteiramente na cache do processador. A segunda é o custo absoluto da remoção na KD-Tree, que se mostrou de uma a duas ordens de magnitude superior ao das demais operações da própria estrutura

(Seção V) – a teoria prevê corretamente que essa operação é assintoticamente mais cara ($O(n^{1-1/k})$ contra $O(\log n)$ das demais), mas a magnitude prática do efeito, amplificada pelo número de dimensões testado, superou a expectativa inicial dos autores, reforçando a relevância prática (e não apenas teórica) da maldição da dimensionalidade discutida na Seção IV.

E. Situações de maior adequação de cada estrutura

Os resultados experimentais corroboram as recomendações teóricas de uso: a AVL é preferível quando garantias de pior caso são essenciais e o padrão de acesso é uniforme; a Splay é vantajosa sob padrões de acesso com forte localidade temporal, como caches, mesmo com maior número absoluto de rotações; a Treap oferece a menor complexidade de implementação entre as estruturas balanceadas, ao custo de uma garantia apenas probabilística; a Trie é indicada quando o alfabeto é pequeno e buscas por prefixo são frequentes; a Patricia é preferível à Trie quando a economia de memória é crítica ou as chaves compartilham grandes prefixos comuns; e a KD-Tree é a única opção viável entre as estudadas para problemas genuinamente multidimensionais, desde que a distribuição espacial dos dados não seja fortemente degenerada. A BST, por sua vez, mostra-se adequada apenas em contextos de prototipagem ou uso educacional, ou quando há garantia externa de que a ordem de inserção será suficientemente aleatória – condição frequentemente difícil de assegurar em sistemas de produção, o que limita sua adequação prática em relação às demais estruturas estudadas, apesar de seu desempenho superior sob entrada favorável (Seção V).

F. Limitações e melhorias possíveis

A implementação atual da Trie e da Patricia restringe o alfabeto suportado a letras minúsculas sem acentuação, exigindo normalização prévia das chaves de entrada – uma limitação de projeto documentada, que poderia ser superada com uma representação baseada em mapa (*hash map*) de filhos em vez de vetor fixo, ao custo de maior overhead constante por acesso. Algumas funções, sobretudo em procurar o sucessor, privilegiaram a legibilidade do código em detrimento do desempenho, realizando travessias adicionais e desnecessárias da árvore durante a operação de remoção – um gargalo identificado apenas na análise de desempenho da Seção V, e que poderia ser eliminado com uma reestruturação direcionada dessa função específica, sem alterar sua interface pública. Por fim, a implementação da KD-Tree poderia ser estendida com uma variante de reconstrução periódica (*rebalancing*) para mitigar a degeneração observada sob dados fortemente correlacionados.

VII. CONCLUSÃO

Este trabalho implementou e avaliou experimentalmente sete estruturas de árvore – BST, AVL, Splay, Treap, KD-Tree, Trie e Patricia –, cujas principais diferenças residem no critério de comparação por nível, no mecanismo (ou ausência) de balanceamento e na granularidade da chave armazenada.

Os resultados foram, em geral, coerentes com a análise teórica: BST e KD-Tree degeneraram para custo quadrático sob entradas adversariais, enquanto as estruturas balanceadas (AVL, Splay, Treap) mantiveram-se em $O(n \log n)$. A principal divergência ocorreu na Splay, cujo desempenho sob entrada ordenada superou o da AVL apesar de número comparável de rotações – explicado pela natureza amortizada, não instantânea, de sua garantia. Sob distribuição aleatória, a BST foi a mais rápida das quatro árvores binárias (sem overhead de verificação estrutural), invertendo-se sob entrada ordenada; a AVL teve o menor tempo de busca; a Splay foi a mais custosa em acesso uniforme, mas a mais vantajosa sob localidade temporal; e a KD-Tree, competitiva em busca/inserção, teve o maior custo absoluto do trabalho na remoção, agravado pelo número de dimensões.

Patricia e KD-Tree foram as de maior dificuldade de implementação – pela manipulação de rótulos de tamanho variável e pela generalização de invariantes para múltiplas dimensões, respectivamente –, exigindo testes dirigidos a casos de borda para revelar defeitos ausentes no caso médio. Quanto à adequação: AVL serve onde previsibilidade de pior caso é essencial (ao custo de maior overhead constante); BST, apenas sob garantia de entrada aleatória (sem qualquer proteção contra degeneração); Splay, sob acesso desigual; Treap, quando simplicidade de implementação com garantia probabilística basta; Trie, para alfabetos pequenos e buscas por prefixo; Patricia, quando memória é crítica; e KD-Tree, para problemas multidimensionais sem forte correlação espacial. Não há estrutura universalmente superior: a escolha depende do compromisso entre garantia teórica, custo constante de implementação e padrão de acesso esperado.

Como trabalho futuro, sugerem-se a extensão do alfabeto de Trie/Patricia, reconstrução periódica da KD-Tree sob distribuições adversariais, e métricas de localidade de cache complementando a análise de comparações e rotações aqui apresentada.

O código-fonte completo das sete implementações, os scripts de geração de dados e gráficos, a suíte de testes utilizada na validação e os resultados brutos dos experimentos estão disponíveis publicamente no repositório do projeto [7].

REFERÊNCIAS

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.
- [2] E. Fredkin, “Trie memory,” *Communications of the ACM*, vol. 3, no. 9, pp. 490–499, 1960.
- [3] D. R. Morrison, “PATRICIA – Practical Algorithm To Retrieve Information Coded in Alphanumeric,” *Journal of the ACM*, vol. 15, no. 4, pp. 514–534, 1968.
- [4] D. D. Sleator and R. E. Tarjan, “Self-Adjusting Binary Search Trees,” *Journal of the ACM*, vol. 32, no. 3, pp. 652–686, 1985.
- [5] R. Seidel and C. R. Aragon, “Randomized Search Trees,” *Algorithmica*, vol. 16, no. 4–5, pp. 464–497, 1996.
- [6] J. L. Bentley, “Multidimensional Binary Search Trees Used for Associative Searching,” *Communications of the ACM*, vol. 18, no. 9, pp. 509–517, 1975.
- [7] R. Mariê, “Estruturas em Árvores Avançadas,” *GitHub repository*, 2026. [Online]. Disponível em: <https://github.com/Ritaa-Marie/Trabalho-Arvores-AEDES2>. [Acesso em: 19 set. 2026].